Prédire la note d'un logement Airbnb à partir du texte des avis

Projet de fouille de données textuelles, Master 2 MAS (SDD), Université Rennes 2

DELAPORTE Noa et SCALART Sarah, le 18/12/2026.

Introduction

Les plateformes de location comme Airbnb reposent en grande partie sur les avis laissés par les voyageurs. En plus d'une note globale, chaque logement est évalué selon plusieurs critères, mais ces notes chiffrées ne disent pas tout : c'est souvent dans le texte des commentaires que les voyageurs expliquent ce qui les a satisfaits ou déçus. Dès lors, il s'agit de savoir si le contenu textuel des avis voyageurs permet de prédire si un logement Airbnb est bien ou moins bien noté sur la propreté, l'emplacement et le rapport qualité-prix, et quels prétraitements linguistiques rendent cette prédiction plus fiable.

Pour répondre à cette question, nous avons suivi une démarche en plusieurs étapes. Nous avons d'abord constitué et nettoyé un jeu de données d'avis associés aux notes des logements. Nous avons ensuite appliqué différents prétraitements linguistiques, puis transformé les textes en représentations vectorielles exploitables par des méthodes de fouille. Enfin, nous avons évalué les modèles obtenus et analysé leurs résultats, en comparant l'effet des prétraitements sur la qualité des prédictions. À chaque étape, nous justifions nos choix et présentons les principales difficultés rencontrées ainsi que les solutions retenues.

1. Problématique

1.1 Pourquoi ce sujet ?

Les notes attribuées sur Airbnb sont très élevées et donc peu discriminantes : dans chacune des catégories, la note médiane tourne autour de 4,8 sur 5. Une note seule permet ainsi difficilement de distinguer un bon logement d'un logement simplement correct. A l'inverse, le texte des avis est beaucoup plus riche : les voyageurs y décrivent précisément ce qui leur a plu ou déplu, souvent avec des nuances que la note ne capture pas ("bien situé mais..", "un peu bruyant"). C'est cet écart entre une note et un texte qui nous a donné envie d'étudier ce que le contenu des avis révèle réellement de la qualité d'un logement.

Ce sujet présente aussi l'intérêt de mesurer concrètement l'apport des traitements linguistiques vus en cours : en comparant plusieurs prétraitements sur une même tâche de prédiction, nous pouvons observer leur effet réel sur les résultats, ce qui correspond à l'objectif principal du projet.

1.2 Choix des catégories étudiées

Nous avons retenu trois catégories de notes qui renvoient à des aspects différents du séjour :

- la propreté, qui porte sur l'état du logement ;
- l'emplacement, qui concerne le quartier, l'environnement et l'accès aux transports ;
- le rapport qualité-prix, qui relève d'une appréciation plus globale et plus subjective.

Ces trois dimensions permettent de voir si la prédiction est plus facile pour les aspects concrets que pour les aspects plus subjectifs.

2. Données

2.1 Choix du jeu de données

Nous utilisons les données publiques d'Inside Airbnb (https://insideairbnb.com/) pour Bordeaux (extraction du 17/09/2026) :

- "reviews.csv" : 516 990 avis portant sur 10 254 logements, avec la date et le texte de l'avis ;
- "listings.csv" : une ligne par logement, avec notamment le quartier, le type de logement et les notes moyennes par catégorie.

Ce jeu de données a été retenu pour plusieurs raisons :

- ce sont des données réelles et volumineuses ;
- la majorité des avis sont en français, ce qui correspond aux outils vus en cours ;
- le texte est bruité, ce qui justifie un vrai travail de nettoyage : balises HTML, émojis, fautes d'orthographe et d'accent, avis écrits dans plusieurs langues ;
- les notes par catégorie fournissent directement des étiquettes pour une classification supervisée.

En revanche, il faut noter que, "reviews.csv" ne contient aucune note par avis. Les notes viennent de "listings.csv" et sont données au niveau du logement : tous les avis d'un même logement portent donc la même note. Ce point a guidé la construction du jeu de données final (partie 4).

2.2 Nettoyage par expressions régulières (notebook 01_nettoyage)

Avant tout traitement, nous avons analysé le corpus brut afin de repérer les éléments à nettoyer. Cette analyse a mis en évidence plusieurs problèmes. Quelques avis étaient totalement vides (8) et d'autres avis ne contenaient que de la ponctuation, comme un simple point (1 052). Le texte comportait aussi de nombreux restes de mise en page du site avec environ 159 000 avis qui contenaient des balises HTML `<br>`. Nous avons également relevé 6 avis contenant une adresse web et près de 93 000 avis utilisant des apostrophes ou des guillemets. Enfin, 22 680 avis contenaient au moins un émoji, pour un total de 29 867 émojis.

À partir de ce constat, nous avons appliqué une série de traitements dans un ordre précis. Nous avons d'abord supprimé la colonne reviewer_name car il s'agit d'une donnée personnelle inutile à l'analyse. Les balises `<br>` ont ensuite été remplacées par un espace et les entités HTML décodées, puisqu'elles ne relèvent pas du texte écrit par les voyageurs. Le texte a été mis en minuscules afin que "Parfait" et "parfait" soient reconnus comme le même mot.

Les adresses web ont été remplacées par le jeton URLEXPR, comme vu au TD5 car une adresse est unique et n'apporte aucune information utile. Pour les émojis, nous avons fait le choix de ne pas les supprimer. Chacun a été remplacé par son nom en français, par exemple emoji_pouce_vers_le_haut. Cela nous a permis de conserver le sentiment exprimé par les voyageurs.

Nous avons ensuite normalisé les apostrophes et les guillemets pour qu'un mot comme "l'hôte" s'écrive de la même façon dans tout le corpus. Les espaces et les retours à la ligne ont aussi été uniformisés. Enfin, nous avons supprimé les avis ne contenant ni lettre, ni chiffre, ni émoji. Les avis

très courts comme un simple pouce levé ou "10/10" ont en revanche été conservés car ils expriment tout de même une opinion.

Après ce nettoyage, le corpus ne contient plus aucune balise, adresse web ou caractère typographique. Il compte au final 515 927 avis.

2.3 Filtrage du corpus (notebook 02_filtrage)

Après le nettoyage, nous avons appliqué plusieurs filtres successifs pour ne garder que des avis exploitables. Le tableau ci-dessous résume l'effet de chaque étape sur la taille du corpus.

Étape	Avis restants	Avis retirés
Après nettoyage	515 927	
Avis recopié sur plusieurs logements (même identifiant)	514 439	1 488
Avis de moins de 5 mots	477 993	36 446
Copies d'un même avis dans un même logement	477 902	91
Avis dans une autre langue que le français	337 787	140 115

Un même avis peut être recopié par Airbnb sur les différents logements d'un même hôtel. Nous n'en avons gardé qu'un exemplaire. Les avis de moins de 5 mots ont été retirés car un avis comme "parfait" ou "très bien" ne dit rien sur un aspect précis du logement. Ce seuil reste prudent puisque le nombre médian de mots par avis est de 23 (voir la figure de la partie 2 "Longueur minimale" du notebook).

Les copies exactes d'un avis dans un même logement ont été supprimées. Les textes identiques sur des logements différents ont en revanche été conservés car ils viennent de voyageurs différents. Enfin, nous n'avons gardé que les avis en français, qui représentent 71 % du corpus contre 21 % pour l'anglais. Cela permet d'utiliser un seul modèle linguistique et une seule liste de mots vides. La langue a été détectée avec la bibliothèque lingua, fiable sur les textes courts.

Chaque avis a ensuite été associé à son logement grâce à l'identifiant listing_id. Cela nous permet de récupérer le quartier, le type de logement et les notes par catégorie. Tous les avis ont trouvé leur logement correspondant.

2.4 Problèmes rencontrés et solutions

Le premier problème est venu de la détection de langue. Comme les émojis ont été remplacés par leur nom en français, un avis anglais comme "great stay" suivi d'un émoji souriant risquait d'être détecté comme français. Pour éviter cette erreur, les jetons émoji sont retirés du texte avant la détection de langue.

Les avis courts comme "parfait" sont souvent identiques par coïncidence. Pour ne pas les confondre avec de vrais doublons, le seuil de longueur est appliqué avant la suppression des copies.

Enfin, la bibliothèque emoji s'est révélée lente sur plus de 500 000 avis. Nous avons ajouté un premier test rapide qui repère les avis susceptibles de contenir un émoji. Seuls ces avis passent ensuite par la

bibliothèque, ce qui divise le temps de traitement par cinq environ.

3. Prétraitements linguistiques (notebook 03_pretraitements_linguistiques)

Cette étape est le coeur du projet. L'objectif est de ne garder que les mots qui portent une opinion ou décrivent un aspect du logement, sans perdre les nuances.

3.1 Outil utilisé

Nous utilisons spaCy avec le modèle français "fr_core_news_md". Il fournit la tokenisation, la lemmatisation, les catégories grammaticales et la reconnaissance des entités nommées. L'analyse syntaxique n'est pas utile ici et a été désactivée pour accélérer le traitement. Tous les avis sont traités en une seule passe avec "nlp.pipe", ce qui prend environ 20 minutes pour les 337 787 avis. Les réglages ont d'abord été mis au point sur un échantillon de 30 000 avis.

3.2 Lemmatisation

La lemmatisation ramène chaque mot à sa forme de base : "situé" et "situés" deviennent "situer", "propres" devient "propre". Elle regroupe les différentes formes d'un même mot, ce qui réduit le vocabulaire et renforce chaque variable du modèle.

3.3 Mots vides adaptés au problème

La liste de mots vides par défaut de spaCy (507 mots) supprime des mots qui changent le sens d'un avis. Avec elle, "pas propre", "très propre" et "peu propre" deviennent tous "propre".

Nous avons donc retiré de cette liste les négations ("ne", "pas", "sans"), les intensifieurs ("très", "peu", "assez", "plus", "moins"), les mots d'opposition ("mais", "cependant") et certains mots d'évaluation comme "proche", utile pour décrire l'emplacement.

À l'inverse, nous avons ajouté les mots du domaine très fréquents qui ne disent rien de la qualité du logement. Par exemple "logement" apparaît dans 34 % des avis, "séjour" dans 31 % et "appartement" dans 28 %. Les fautes d'accent courantes comme "tres" ou "hote" ont aussi été ajoutées aux listes.

3.4 Gestion de la négation

Pour un modèle fondé sur les mots, "pas propre" et "propre" contiennent tous les deux le mot "propre". Nous transformons donc le mot nié en un nouveau mot. "pas propre" devient "NEG_propre", que le modèle peut apprendre comme une variable distincte.

Nous avons choisi une règle simple plutôt qu'une analyse syntaxique, plus lente et pas plus fiable sur des avis souvent mal écrits. Un mot de négation ("ne", "pas", "jamais", "rien", "sans", "aucun") porte sur le premier mot plein qui suit, c'est-à-dire un nom, un adjectif, un verbe ou un adverbe. Les mots-outils sont sautés, ainsi "pas de problème" donne "NEG_problème". La négation traverse aussi les

intensifieurs et "pas très propre" donne "NEG_très NEG_propre". Elle s'arrête à une ponctuation ou à un mot comme "mais" ou "et".

Le tableau suivant montre trois exemples obtenus sur des avis réels.

Avis	Texte après traitement
"La douche ne fonctionnait pas. Merci Marie !"	"douche NEG_fonctionner HOTE"
"La télévision ne correspond pas du tout à l'annonce."	"télévision NEG_correspondre annonce"
"Rien à redire sur l'emplacement, proche du tram. Appartement sans charme."	"NEG_redire emplacement proche tram NEG_charme"

Sur tout le corprus 16,8 % des avis contiennent au moins une négation.

3.5 Remplacement des prénoms par un jeton (entités nommées)

Les avis citent souvent l'hôte par son prénom ("merci à Julien"). Ces prénoms sont propres à un seul logement et favoriseraient le surapprentissage. Nous les remplaçons donc par le jeton "HOTE", ce qui protège aussi la vie privée des hôtes.

Les prénoms sont repérés grâce aux entités nommées de spaCy (personnes étiquetées comme noms propres) et à la colonne "host_name" du fichier "listings.csv". Sur le corpus, 41,5% des avis citent au moins un prénom.

3.6 Filtre grammatical

Nous ne gardons que les noms, noms propres, adjectifs, verbes et adverbes, qui portent l'essentiel des opinions et des aspects. Les noms propres sont conservés car "Parfait" en début d'avis est souvent étiqueté ainsi et les noms de quartiers renseignent sur l'emplacement.

3.7 Versions du texte pour l'étude d'ablation

Pour mesurer l'apport de chaque traitement, nous produisons plusieurs versions du texte. Chacune ajoute un seul traitement à la précédente. Le tableau suivant les présente sur un même exemple, ce qui permet de voir directement l'effet de chaque étape.

Version	Traitement ajouté	Résultat
"comments_clean"	texte nettoyé, sans traitement linguistique	"le logement n'était pas très propre mais bien situé, julien est super"
"text_lemmes"	lemmes et mots vides par défaut de spaCy	"logement propre bien situer HOTE super"
"text_stop"	mots vides adaptés	"ne pas très propre mais bien situer HOTE super"
"text_neg"	négation	"NEG_très NEG_propre mais bien situer HOTE super"
"text_pos"	filtre grammatical	"NEG_très NEG_propre bien situer HOTE super"

Dans toutes les versions lemmatisées, les prénoms sont remplacés par "HOTE" et les jetons émoji sont conservés.

3.8 Problèmes rencontrés et solutions

La liste de mots vides par défaut supprimait "pas", "très" et "mais", d'où la création de notre propre liste. Les jetons émoji étaient aussi retirés par le filtre alphabétique à cause du caractère "_". Nous les avons donc conservés explicitement.

La reconnaissance des prénoms s'appuie sur la majuscule. spaCy travaille donc sur un texte non mis en minuscules et les lemmes sont mis en minuscules ensuite. Pour éviter que "Bonne" ou "Bel" en début de phrase soient pris pour des prénoms, les entités "PER" sont limitées aux noms propres.

La négation produisait des erreurs. "rien à dire, tres bonne adresse" donnait "NEG_bon" et "à deux pas du centre" était pris pour une négation. La règle du premier mot plein et la lecture d'exemples réels ont permis de corriger ces cas.

Enfin, les accents ne sont retirés qu'à l'étape de représentation car spaCy en a besoin pour lemmatiser.

4. Représentation des documents (notebook 04_representation_vectorielle)

4.1 Un document par logement

Les notes sont données au logement et non à l'avis. Avec un avis par ligne, les avis d'un même logement se seraient retrouvés à la fois dans l'entraînement et dans le test. Le modèle aurait alors pu reconnaître le logement au lieu d'apprendre ce qui le rend propre ou bien situé.

Nous avons donc regroupé tous les avis d'un logement en un seul document. Chaque ligne correspond ainsi à un logement avec un texte et une note. Seuls les logements ayant au moins 10 avis en français sont conservés, car en dessous le texte est trop court pour juger un aspect. La table finale compte 5 497 logements.

4.2 Construction des étiquettes

Les notes médianes sont très élevées : 4,83 pour la propreté, 4,79 pour l'emplacement et 4,76 pour le rapport qualité-prix. Un seuil fixe comme 4 sur 5 ne laisserait presque aucun logement dans la classe "moins bien noté".

Pour chaque catégorie, nous comparons donc le quart des logements le moins bien noté (classe 0) au quart le mieux noté (classe 1). La moitié du milieu est écartée car l'écart entre ces logements est trop faible pour apparaître dans le texte. Le tableau suivant résume les seuils et les effectifs obtenus.

	Propreté	Emplacement	Qualité-prix
Seuil de la classe 0 (premier quartile)	4,68	4,69	4,64
Seuil de la classe 1 (troisième quartile)	4,93	4,89	4,85
Logements en classe 0	1 423	1 461	1 451
Logements en classe 1	1 482	1 389	1 475
Logements écartés	2 592	2 647	2 571

Les deux classes sont équilibrées dans chaque catégorie. La propreté et le rapport qualité-prix sont fortement corrélés (0,72). L'emplacement l'est beaucoup moins avec la propreté (0,34) et avec le rapport qualité-prix (0,46).

4.3 Représentation TF-IDF

Chaque document est représenté par un vecteur TF-IDF, comme au TD5. Un mot pèse davantage s'il est fréquent dans le document et rare dans les autres. Un mot présent partout pèse donc peu.

Les accents sont supprimés à cette étape pour que "très" et "tres" deviennent le même mot. Les mots présents dans moins de 5 logements sont ignorés, car il s'agit souvent de fautes de frappe ou de noms rares. Nous comparons deux configurations : les mots seuls, puis les mots seuls avec les paires de mots consécutifs. Ces paires gardent un peu de contexte, par exemple "très propre" ou "peu propre".

Le TF-IDF est toujours appris sur les données d'entraînement uniquement. Le vocabulaire et les poids ne tiennent donc pas compte des données de test.

Version	unigrammes	uni + bigrammes
comments_clean	12462	122571
text_lemmes	7957	89311
text_stop	7916	91438
text_neg	8744	90366
text_pos	8489	88598

4.4 Problèmes rencontrés et solutions

Une première version gardait un avis par ligne, ce qui provoquait la fuite de données décrite plus haut. Le regroupement par logement l'a corrigée. La cible reposait d'abord sur la médiane de la note globale. Elle a été remplacée par les quartiles de chaque catégorie, plus fidèles à la problématique. Enfin, le TF-IDF était au départ appris sur tout le corpus. Il est désormais intégré au modèle et appris sur l'entraînement seulement.

5. Méthode de fouille (notebook 05_methode_de_fouille)

5.1 Choix de la méthode

Nous disposons d'étiquettes et cherchons à les retrouver à partir du texte. Il s'agit donc d'une classification supervisée binaire, une par catégorie. Comme au TD5, nous comparons quatre modèles :

- un classifieur naïf qui prédit toujours la classe majoritaire, comme référence à battre ;
- Naive Bayes, simple et classique pour le texte ;
- la régression logistique, performante sur du TF-IDF et facile à interpréter ;
- le SVM linéaire ("LinearSVC"), adapté aux grandes matrices creuses.

Chaque modèle est un "Pipeline" scikit-learn qui enchaîne le TF-IDF et le classifieur.

5.2 Protocole d'évaluation

La mesure principale est le F1 macro, qui vérifie que les deux classes sont bien reconnues. Les comparaisons entre modèles reposent sur une validation croisée à 5 plis, plus stable qu'un seul découpage. Le modèle final est évalué sur un découpage stratifié 75 % / 25 % avec une graine fixée, à l'aide de l'accuracy, la précision, le rappel, le F1 et la matrice de confusion. L'étude d'ablation garde le même modèle et les mêmes logements et ne change que la version du texte.

5.3 Problèmes rencontrés et solutions

Lorsque les données comptaient encore un avis par ligne, le SVM classique de scikit-learn ("SVC") était beaucoup trop lent. Nous utilisons donc "LinearSVC", qui produit le même type de modèle bien plus rapidement.

6. Résultats et analyse

6.1 Comparaison des modèles

Le tableau suivant donne le F1 macro en validation croisée pour chaque modèle, sur la version "text_neg" avec unigrammes et bigrammes.

Modèle	Propreté	Emplacement	Qualité-prix
Classe majoritaire (référence)	0,338	0,339	0,335
Naive Bayes	0,819	0,786	0,854
Régression logistique	0,900	0,833	0,878
SVM linéaire	0,912	0,854	0,893

Tous les modèles dépassent très largement la référence, ainsi le texte des avis permet bien de distinguer les logements bien notés des logements moins bien notés. Le SVM linéaire est le meilleur dans les trois catégories et il est retenu pour la suite.

L'emplacement est la catégorie la plus difficile à prédire. Une explication possible est que ses notes sont les plus resserrées : l'écart entre les deux seuils n'est que de 0,20 point, contre 0,21 pour le rapport qualité-prix et 0,25 pour la propreté (partie 4.2). De plus, beaucoup de voyageurs écrivent "bien situé" quel que soit le logement. Un quartier peut aussi être jugé bien ou mal selon les attentes de chacun.

6.2 Étude d'ablation : apport de chaque prétraitement

Le tableau suivant donne la moyenne du F1 macro sur les trois catégories, en validation croisée, avec le SVM linéaire.

Version du texte	Unigrammes	Unigrammes et bigrammes
Texte nettoyé, sans traitement linguistique	0,873	0,883
Lemmes, mots vides par défaut	0,879	0,880
Mots vides adaptés	0,882	0,884
Négation	0,882	0,886
Filtre grammatical	0,882	0,884

Les prétraitements linguistiques améliorent la prédiction, mais de façon modeste : en unigrammes, le score passe de 0,873 pour le texte nettoyé à 0,882 avec les lemmes et les mots vides adaptés. L'essentiel du gain vient de la lemmatisation, qui regroupe les formes d'un même mot. Les mots vides adaptés ajoutent un léger gain. La meilleure version est "text_neg" avec unigrammes et bigrammes (0,886). C'est donc elle que nous retenons pour la suite, même si l'écart avec "text_stop" (0,884) est trop faible pour conclure.

La négation et le filtre grammatical n'apportent donc pas de gain net. Les écarts sont inférieurs à 0,005, trop faibles pour conclure à une différence. Plusieurs raisons peuvent l'expliquer. Les bigrammes conservent déjà des expressions comme "pas propre". Les négations les plus fréquentes sont aussi souvent positives, comme "sans hésiter" (présent dans 3 428 logements) ou "rien à redire" (2 178 logements). Enfin, chaque document regroupe de nombreux avis, si bien que quelques négations pèsent peu dans l'ensemble.

Les bigrammes améliorent surtout le texte nettoyé (+0,009) et beaucoup moins les versions lemmatisées. Avec les bigrammes, le texte nettoyé (0,883) fait même presque aussi bien que la meilleure version : une bonne partie de l'information est déjà présente dans le texte sans traitement linguistique.

6.3 Évaluation détaillée du modèle final

Le modèle final est le SVM linéaire appliqué à "text_neg" avec unigrammes et bigrammes. Il est évalué sur 25 % des logements, jamais vus pendant l'entraînement.

Catégorie	Accuracy	Précision	Rappel	F1
Propreté	0,898	0,899	0,898	0,898
Emplacement	0,843	0,843	0,843	0,843
Qualité-prix	0,883	0,883	0,882	0,882

La précision indique la part des logements classés "moins bien notés" qui le sont vraiment. Le rappel indique la part des logements moins bien notés que le modèle retrouve. Le F1 combine les deux. Ces mesures sont calculées pour chaque classe puis moyennées (moyenne macro).

Les scores obtenus confirment ceux de la validation croisée. Le modèle reconnaît correctement environ 9 logements sur 10 pour la propreté et le rapport qualité-prix, et un peu plus de 8 sur 10 pour l'emplacement. Pour chaque classe prise séparément, la précision et le rappel restent compris entre 0,83 et 0,91, et les erreurs se répartissent de façon proche entre les deux classes (par exemple 42 et 32 erreurs pour la propreté). Le modèle ne privilégie donc aucune classe.

6.4 Mots les plus associés à chaque classe

Les coefficients d'un modèle linéaire indiquent les termes qui poussent vers chaque classe : un coefficient positif pousse vers "mieux noté", un coefficient négatif vers "moins bien noté". On vérifie ainsi que le modèle s'appuie sur des mots qui parlent vraiment de l'aspect étudié.

Principaux termes obtenus avec le SVM linéaire sur "text_neg" (les accents sont retirés par le TF-IDF) :

Catégorie	Moins bien noté	Mieux noté
Propreté	"bon", "mais", "situer", "peu", "bien situer", "pratique", "menage", "bien", "agreable", "odeur", "sympa", "sale", "prix", "arranger", "cependant"	"propre", "tres propre", "impeccable", "parfait", "accueil", "neuf", "revenir", "decorer", "propre bien", "excellent", "gout", "magnifique", "gentillesse", "plaisir", "equipe"
Emplacement	"bon", "peu", "bus", "passer", "marche", "capucin", "gare", "mais", "agreable", "correspondre", "photo", "velo", "assez", "marche capucin", "bruyant"	"situer", "emplacement", "placer", "tres bien", "idealement", "calme", "bien placer", "excellent", "idealement situer", "paisible", "parfait", "plein centre", "revenir", "bien situer", "restaurant"
Qualité-prix	"peu", "mais", "bon", "conforme", "situer", "bien situer", "probleme", "sympa", "cependant", "bemol", "mois", "jacuzzi", "photo", "bien", "dommage"	"hote", "excellent", "confortable", "parfait", "revenir", "super", "recommander", "impeccable", "attentionne", "neg_hesiter", "recommander neg_hesiter", "decorer", "beaucoup", "plaisir", "propre"

Ces termes montrent que le texte permet bien de retrouver chaque aspect du logement. Pour la propreté, "sale", "odeur" et "menage" signalent les logements moins bien notés, "propre" et "impeccable" les mieux notés. Pour l'emplacement, "plein centre", "calme" et "paisible" s'opposent à "gare", "bus" et "bruyant". Quelques termes comme "jacuzzi" ou "mois" sont plus difficiles à interpréter et viennent sans doute d'un petit nombre de logements.

Les mots de nuance comme "mais", "peu", "cependant" ou "bemol" sont de forts indices d'un logement moins bien noté, ce qui confirme l'intérêt de les avoir conservés. De même, "bon" est un compliment plus tiède que "parfait". Le terme "bien situer" pousse vers "mieux noté" pour l'emplacement, mais vers "moins bien noté" pour la propreté et le rapport qualité-prix : seul, il suggère que le reste du séjour a moins convaincu.

Les termes "neg_hesiter" et "recommander neg_hesiter", issus de "sans hésiter", font partie des indices d'un logement mieux noté pour le rapport qualité-prix. Le marquage de la négation transforme cette expression positive en un mot "NEG_", mais le modèle a bien appris qu'elle était positive.

Le jeton "hote" remplace le prénom de l'hôte. Il indique un contact personnel associé aux logements mieux notés. Il arrive en tête pour le rapport qualité-prix et des mots comme "accueil", "gentillesse" ou "revenir" ressortent aussi pour la propreté. Le modèle capte donc aussi une satisfaction générale, en particulier pour ces deux catégories très corrélées. L'emplacement garde un vocabulaire plus spécifique.

6.5 Réponse à la problématique

Le contenu textuel des avis permet bien de prédire si un logement est bien ou moins bien noté. Le SVM linéaire obtient un F1 macro de 0,91 pour la propreté, 0,89 pour le rapport qualité-prix et 0,85 pour l'emplacement en validation croisée, très loin de la référence à 0,34. Les mots utilisés par le modèle correspondent bien à chaque aspect, même si une part de satisfaction générale se mêle aux catégories propreté et rapport qualité-prix.

Les prétraitements les plus utiles sont la lemmatisation et les mots vides adaptés, qui font passer le score de 0,873 à 0,882 en unigrammes. Le gain reste toutefois modeste, car le texte nettoyé contient déjà l'essentiel de l'information. Garder les intensifieurs et les mots de nuance s'est surtout révélé utile pour interpréter le modèle, puisqu'ils figurent parmi les termes les plus discriminants. Le marquage de la négation donne le meilleur score (0,886), mais l'écart est trop faible pour conclure à un vrai gain, et le filtre grammatical n'apporte pas de gain mesurable dans notre cas.

D'un point de vue métier, les avis contiennent une information que les notes, toutes proches de 5, ne reflètent plus. Airbnb pourrait s'en servir pour repérer plus tôt les logements à problème, par exemple ceux dont les avis évoquent une odeur ou un ménage insuffisant. Les hôtes y trouveraient aussi des pistes concrètes d'amélioration.

7. Limites

Les notes sont attribuées au logement et non à chaque avis. Un avis négatif sur un logement bien noté reçoit donc l'étiquette "mieux noté". Le découpage en quartiles écarte aussi la moitié des logements, si bien que le modèle ne dit rien sur les logements moyens.

Les règles de négation et de détection des prénoms restent simples. Par exemple "sans hésiter" devient "NEG_hésiter" alors que l'expression est positive, même si le modèle apprend ensuite à la lire comme un indice positif (partie 6.4). Les écarts de l'étude d'ablation sont par ailleurs faibles et peuvent venir du hasard du découpage.

Enfin, l'étude ne porte que sur les avis en français de Bordeaux. Le TF-IDF ignore aussi l'ordre des mots au-delà des bigrammes, ce qu'un modèle comme CamemBERT pourrait mieux capter.

8. Conclusion et ressenti

Ce projet montre que le texte des avis suffit à distinguer les logements bien et moins bien notés, à condition d'adapter les prétraitements au problème posé.

L'analyse textuelle a fait ressortir des résultats que nous n'attendions pas. Un simple "bon" signale plutôt un logement moins bien noté et des mots comme "mais" ou "peu", que l'on aurait supprimés par défaut, comptent parmi les plus discriminants. On a souvent tendance à faire davantage confiance aux chiffres qu'au texte. Pourtant, dans le cas des avis Airbnb, c'est le texte qui explique ce que les voyageurs ont apprécié ou non, là où les notes sont presque toutes identiques.

Nous avons aussi appris qu'un réglage par défaut peut effacer l'information utile et qu'un traitement plus élaboré n'améliore pas forcément les résultats. Seule une étude d'ablation permet de le vérifier.

Pour aller plus loin, nous pourrions analyser les avis phrase par phrase afin d'isoler chaque aspect. Un modèle comme CamemBERT permettrait aussi de mieux tenir compte du contexte des avis.

Annexe : organisation du code et exécution

Notebook	Étape	Fichier produit
"01_nettoyage.ipynb"	Nettoyage par expressions régulières	"reviews_clean.parquet"
"02_filtrage.ipynb"	Filtrage et jointure avec les logements	"reviews_fr.parquet"
"03_pretraitements_linguistiques.ipynb"	Prétraitements linguistiques avec spaCy	"reviews_ablation.parquet"
"04_representation_vectorielle.ipynb"	Un document par logement, représentation TF-IDF	"logements.parquet"
"05_methode_de_fouille.ipynb"	Classification, étude d'ablation, analyse	tableaux de résultats dans "resultats"

Les fonctions utilisées par les notebooks sont regroupées dans le dossier "src" : "nettoyage.py", "filtrage.py", "pre_traitement.py", "agregation.py" et "fouille.py". Les bibliothèques nécessaires sont listées dans "requirements.txt", et l'ordre d'exécution est décrit dans le fichier "README.md".